Candidate Brief | Fruit Machine Take-Home

👋 Welcome, and thanks for taking the time. This is a take-home exercise followed by a **60-minute review session** with two of our engineers. Please read this page in full before you start: it tells you exactly what to build, how we want it submitted, and how we will assess it. There are no trick questions here.

Timeline

You have **one week** to send your solution back to us, so you can fit it around your schedule. We expect the work itself to take **no more than 4 hours**: please don't feel you need to spend longer. We are not looking for a polished product, and we would much rather see a focused, well-reasoned core than something over-built. If you run out of time, submit what you have and note in your README what you would do next. Prioritising well within the time is itself part of what we're looking at.

What you'll build

You are building a virtual **Fruit Machine**: a small, self-contained program (no external services or APIs required). It has slots that show colours when you spin, and it pays out prizes based on the result. Build it up in the three parts below. Each part builds on the previous one.

Build it as a **Spring Boot service driven through REST endpoints** (this is a backend role, so we want to see a little infrastructure around the game logic, not just a library). **Java or Kotlin** are both fine. Structure it as a real project you would be happy for a teammate to pick up.

Part 1 — Core game logic

The machine has **four slots**. Each slot can display one of four colours: **black, white, green, or yellow**.

- A **spin** randomly selects a colour for each of the four slots.
- Detect a **jackpot**: all four slots show the same colour.
- Return the outcome of the play.

Part 2 — Payouts and prizing

The machine holds a **float** (a starting sum of money). A single play has a fixed **cost**. After a spin, update and return the float according to these rules:

- **Jackpot** (all four slots the same colour): pays out the **entire** float.
- **Full house** (all four slots different colours): pays out **half** the float.
- **Small prize** (two or more **adjacent** slots the same colour): pays out **five times** the cost of a single play.
- If a prize is won but the float is too small to pay it in full, credit the player with **free plays** equal to the shortfall. This does **not** apply to a jackpot.

Part 3 — Generalise and scale

Generalise the machine so it is not hard-wired to four slots and four colours:

- The **number of slots** is variable.
- There can be **hundreds of colours**.
- A **small prize** is now any `k` adjacent slots of the same colour, where `k` is an input (the original game is `k = 2`).

Aim for a solution that stays efficient as the number of slots grows.

Part 4 — Expose it as a service

Wrap the machine in a **Spring Boot service** and drive it through **REST endpoints**. The exact shape is your call, but we would expect something along the lines of:

- Play the machine (spin, evaluate, update the float, return the outcome and any free plays credited).
- Read the current machine state (float, configuration).
- Configure a machine (slot count, colours, `k`, play cost, starting float).

Think about request/response design, input validation, error handling, and where the machine's **state** lives between calls. Keep the game logic cleanly separated from the web layer.

🧩

Where the spec is ambiguous, that is deliberate. Make a reasonable decision, and write down what you decided and why. We are more interested in your judgement than in a single "correct" reading.

What we require

- A complete, working solution. We are looking for a finished piece of work, not a sketch.
- Automated tests. These are required, not optional. We want to see what you chose to test and how.
- A public repository we can review (GitHub, GitLab, Bitbucket, your choice).
- Commit history that shows how the solution came together (please don't squash everything into a single commit).
- A short README / decision log (see below).
- An AI usage log. A short record of how you used AI on this: the prompts or approach that worked well, where it went wrong, and where you steered it or threw its output away. We find this genuinely interesting, and it shows us how you work with these tools.

The README / decision log

- How to run the project and the tests.
- The key engineering decisions you made, and why.
- Anything you deliberately left out, and what you would do with more time.
- Where you used AI, and any point where you disagreed with it or reworked what it gave you.

Nice to haves (optional)

None of these are required, and we would always rather see a solid core than a thin core surrounded by extras. If you have time and it interests you, any of the following are a nice bonus and give us more to talk about:

- **API documentation**, for example an OpenAPI / Swagger spec or a short endpoints section in the README.
- **Containerisation**, for example a Dockerfile or compose file so we can run it easily.
- **CI**, for example a GitHub Actions workflow that runs your tests.
- **Persistence**, for example storing machine state in a real datastore rather than in memory.
- **Observability**, for example sensible logging or basic metrics around plays and payouts.

Using AI

🤖

Using AI tooling is allowed and encouraged. We use these tools every day and we expect you to. What we are assessing is not whether you used AI, but the **quality of your engineering judgement on top of it.** You should understand and be able to explain **every** decision in your submission, because we will ask you to walk us through it. Code you cannot explain is a much weaker signal than a smaller solution you fully own.

What we're grading

🔍We are **not** grading the code artifact on its own, and we are not looking for one "correct" implementation. We are assessing the **concepts and the thinking behind your solution**: the areas below. Expect us to **dig into each of these in the review session**, so the code is really there to give us something concrete to talk through.

</aside>

What we're assessing	What good looks like
Completeness & correctness	The solution works, handles the rules (including the edge cases), and does what the brief asks.
Code quality	Clear structure, sensible naming, readable, easy for a teammate to follow.
Testing	Meaningful automated tests: the important behaviour and edge cases are covered, and the tests are easy to read.
Extensibility	The design can grow. Adding a new rule or changing a parameter shouldn't mean a rewrite.
Service & API design	Sensible endpoints, clean request/response and validation, game logic kept separate from the web layer, and a sensible approach to state.
Reasoning	You can explain *why* you made the choices you made, and weigh the trade-offs.
Communication	You can walk us through the code and your thinking clearly in the review session.

How to submit

- Push your code to a **public** repository.
- Please name the repo using the convention: `phorest-techtest-${yourName}`.
- Send us the link when you're ready.

Good luck, and have fun with it. If anything in the brief is unclear, make a sensible call, note it, and we'll talk it through in the session.

The follow-up: 30-minute review session - followed by a 30 minute system design step

After you submit, we will sit down together for about 30 mins and go through your solution. This session is where most of our assessment happens, so the goal of the take-home is to give us something real to talk about. Expect us to:

- Ask you to **walk us through your design** and the reasoning behind it.
- Dig into **how you approached testing** and why you tested what you did.
- Explore **how you would extend the code**: we may ask you to make a change or add a feature with us, there and then.
- Talk through the **decisions and trade-offs**, including where you took AI's output in a different direction.

There is nothing to prepare beyond knowing your own submission well. Come ready to explain and to change it with us.